

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DA PARAÍBA
UNIDADE ACADÊMICA DE INFORMAÇÃO E COMUNICAÇÃO
COORDENAÇÃO DO CURSO SUPERIOR DE TECNOLOGIA EM SISTEMAS PARA INTERNET

RELATÓRIO DE ESTÁGIO

Desenvolvimento Full-Stack de um Módulo de Solicitações para Transição de Categoria no Sistema Banco de Especialistas

Luis Kilmer da Silva Bernardo

João Pessoa - PB
10/2026

Instituto Federal de Educação, Ciência e Tecnologia da Paraíba
Unidade Acadêmica de Informação e Comunicação
Coordenação do Curso Superior de Tecnologia em Sistemas para Internet

Desenvolvimento Full-Stack de um Módulo de Solicitações para Transição de Categoria no Sistema Banco de Especialistas

Luis Kilmer da Silva Bernardo

Relatório de Estágio apresentado ao Coordenação do Curso Superior de Tecnologia em Sistemas para Internet da Instituto Federal de Educação, Ciência e Tecnologia da Paraíba, como requisito parcial para a obtenção do título de Tecnólogo.

Orientador:	Juliana Dantas Ribeiro Viana de Medeiros
Supervisor:	Juliana Dantas Ribeiro Viana de Medeiros
Coordenador do Curso:	Cândido José Ramos do Egypto
Empresa:	Agência de Inovação (NEO-IFPB)
Período:	25/02/2026 à 19/06/2026

João Pessoa - PB

APROVAÇÃO

Esta folha deverá ser substituída pela cópia digitalizada da folha de aprovação fornecida.

Juliana Dantas Ribeiro Viana de Medeiros
Supervisor da Empresa

Cândido José Ramos do Egypto
Coordenador do CST de Sistemas para Internet

Prof. Dr. Juliana Dantas Ribeiro Viana de Medeiros
Professor Orientador

Luis Kilmer da Silva Bernardo
Estagiário

Altere este texto inserindo a dedicatória do seu trabalho.

AGRADECIMENTOS

Edite e coloque aqui os agradecimentos às pessoas e/ou instituições que contribuíram para a realização do trabalho.

É obrigatório o agradecimento às instituições de fomento à pesquisa que financiaram total ou parcialmente o trabalho, inclusive no que diz respeito à concessão de bolsas.

Eu denomino meu campo de Gestão do Conhecimento, mas você não pode gerenciar conhecimento. Ninguém pode. O que pode fazer - o que a empresa pode fazer - é gerenciar o ambiente que otimize o conhecimento. (PRUSAK, Laurence, 1997).

RESUMO

Silva Bernardo, Luis Kilmer. Desenvolvimento Full-Stack de um Módulo de Solicitações para Transição de Categoria no Sistema Banco de Especialistas. 10/2026. 20 f. Relatório de Estágio – Coordenação do Curso Superior de Tecnologia em Sistemas para Internet, Instituto Federal de Educação, Ciência e Tecnologia da Paraíba. João Pessoa - PB, 10/2026.

Este relatório apresenta as atividades desenvolvidas no Laboratório Assert do Instituto Federal da Paraíba (IFPB), Campus João Pessoa, entre fevereiro e junho de 2026, por meio do PIAGI. O estágio teve como objetivo desenvolver um módulo para o sistema Banco de Especialistas, visando formalizar e rastrear solicitações de alterações cadastrais e de classificação dos usuários. Para isso, implementou-se o módulo de Solicitações, contemplando a modelagem das entidades Solicitação e Snapshot, as regras de negócio para criação, edição, aprovação e rejeição de solicitações e uma linha do tempo (timeline) para registrar os eventos e as decisões do processo. O desenvolvimento também abrangeu a implementação de operações de persistência e endpoints, a validação e serialização de dados, a integração com a arquitetura existente e a construção de funcionalidades de interface para solicitantes e avaliadores. A solução utilizou Python, Flask, PostgreSQL, SQLAlchemy e Marshmallow no back-end, Nuxt no front-end e Docker e Docker Compose para configuração do ambiente. Também foram desenvolvidos testes automatizados com Pytest para verificar as funcionalidades implementadas. Como resultado, o módulo integrou o fluxo de solicitações à plataforma, reduzindo a dependência de comunicações por e-mail e intervenções manuais no banco de dados, além de proporcionar maior formalização, rastreabilidade e histórico das alterações realizadas.

Palavras-chave: Desenvolvimento full-stack. Banco de Especialistas. Solicitações. Rastreabilidade. Testes automatizados.

LISTA DE FIGURAS

Figura 1 – Fluxo de alteração de categoria de especialistas antes do desenvolvimento do módulo. 10

LISTA DE QUADROS

LISTA DE TABELAS

LISTA DE ALGORITMOS

SUMÁRIO

LISTA DE FIGURAS viii

LISTA DE QUADROS ix

LISTA DE TABELAS x

LISTA DE ABREVIATURAS E SIGLAS xi

LISTA DE SÍMBOLOS xi

LISTA DE ALGORITMOS xi

1 – INTRODUÇÃO 1
- 1.1 Objetivos 1
  - 1.1.1 Objetivo geral 1
  - 1.1.2 Objetivos específicos 2
- 1.2 Escopo e limitações 2
- 1.3 Procedimentos metodológicos 2
- 1.4 Contexto institucional 3
- 1.5 Organização do relatório 3

2 – EMBASAMENTO TÉORICO 4
- 2.1 Arquitetura Web Cliente-Servidor 4
- 2.2 Flask 4
- 2.3 Persistência de dados com PostgreSQL e SQLAlchemy 4
  - 2.3.1 PostgreSQL 5
  - 2.3.2 SQLAlchemy 5
- 2.4 Validação de Dados com Marshmallow 5
- 2.5 Auditoria, Rastreabilidade e Versionamento de Dados 5
  - 2.5.1 Auditoria e rastreabilidade 5
  - 2.5.2 Fluxo de aprovação com máquina de estados 6
  - 2.5.3 Snapshot e reversão 6
  - 2.5.4 Timeline e registro de eventos 6
- 2.6 Vue.js e Nuxt 7
- 2.7 Verificação das Transições com Pytest 7
- 2.8 Ambiente Reprodutível de Testes com Docker 7
- 2.9 Controle de Versão e Gestão de Configuracao 7
- 2.10 Síntese do Capítulo 8

3 – ATIVIDADES REALIZADAS 9
3.1 Estudo da Arquitetura e Levantamento de Requisitos 9
3.2 Análise das Regras de Negócio 9
3.3 Entendimento do Fluxo Antigo 10
3.4 Desenvolvimento do Módulo de Solicitações 11
3.4.1 Endpoints 11
3.4.2 Integração com o banco de dados 11
3.4.3 Aprovação e Rejeição 11
3.4.4 Mudança de categoria 11
3.4.5 Atualização de pontuação 11
3.5 Implementação do Sistema de Snapshot e Auditoria 11
3.5.1 Entidade Snapshot 11
3.5.2 JSONB 11
3.5.3 Reversão 11
3.5.4 Timeline 11
3.5.5 Rastreabilidade 11
3.6 Desenvolvimento das Interfaces Web 11
3.6.1 Telas Criadas 11
3.6.2 Componentes Vue 11
3.6.3 Integração frontend/backend 11
3.7 Testes e Validação 11
3.7.1 Pytest 11
3.7.2 Pré-configuração 11
3.7.3 Testes unitários 11
3.7.4 Testes de integração 11
3.7.5 Processo de Integração Contínua 11
3.8 Levantamento de Requisitos das Funcionalidades 11

4 – CONSIDERAÇÕES FINAIS 12

Referências 13

Apêndices 15

APÊNDICE A – Nome do apêndice 16

APÊNDICE B – Nome do outro apêndice 17

Anexos 18

ANEXO A–Nome do anexo 19

ANEXO B–Nome do outro anexo 20

1 INTRODUÇÃO

O presente relatório descreve as atividades desenvolvidas durante o estágio supervisionado realizado entre fevereiro e junho de 2026 no Laboratório Assert, vinculado ao Polo de Inovação do Instituto Federal da Paraíba (IFPB). As atividades concentraram-se na evolução do sistema Banco de Especialistas, plataforma destinada ao gerenciamento de especialistas e ao apoio à composição de equipes para projetos de Pesquisa, Desenvolvimento e Inovação (PD&I).

O Banco de Especialistas permite o cadastro, a classificação e a avaliação de candidatos com base em critérios como formação acadêmica, experiência profissional e participação em projetos. Nesse contexto, a plataforma reúne informações utilizadas na gestão dos especialistas e nos processos de seleção e composição de equipes vinculadas às iniciativas do Polo de Inovação.

Durante o estágio, identificou-se uma demanda relacionada à alteração de categoria dos especialistas. O procedimento então utilizado exigia que o interessado solicitasse a mudança por e-mail e que um administrador realizasse manualmente uma consulta SQL diretamente no banco de dados. A ausência de um fluxo formal de solicitações dificultava o acompanhamento das alterações e a recuperação do estado anterior dos registros, além de aumentar o risco de inconsistências decorrentes de intervenções diretas na base de dados.

Diante desse cenário, o trabalho concentrou-se no desenvolvimento de um módulo de solicitações para formalizar e gerenciar operações de alteração cadastral e de classificação dos especialistas. A solução contempla mecanismos para registrar solicitações, acompanhar seu andamento, permitir sua análise e decisão e manter informações necessárias à rastreabilidade das operações. Dessa forma, busca-se reduzir a dependência de intervenções manuais no banco de dados e estabelecer um processo mais controlado e verificável.

Do ponto de vista técnico, o desenvolvimento envolveu conceitos de engenharia de software, aplicações web, persistência de dados relacionais, testes automatizados e conteinerização. Destacam-se a integração entre componentes de front-end e back-end, o uso de mecanismos de registro e recuperação do estado dos usuários e a manutenção de um histórico das operações. Esses fundamentos são apresentados no capítulo dedicado ao embasamento teórico.

1.1 Objetivos

1.1.1 Objetivo geral

Desenvolver um módulo de solicitações de alteração cadastral e de classificação no sistema Banco de Especialistas, contemplando os fluxos de criação, consulta, aprovação, rejeição e reversão, com mecanismos de rastreabilidade e registro das operações.

1.1.2 Objetivos específicos

- Levantar e analisar os requisitos e as regras de negócio relacionados às solicitações de alteração cadastral e de classificação dos especialistas.
- Modelar os componentes de persistência necessários ao registro das solicitações e à recuperação do estado dos usuários.
- Implementar as funcionalidades de gerenciamento das solicitações no back-end, incluindo as regras de negócio, a integração com o PostgreSQL e o acesso aos dados por meio do SQLAlchemy.
- Desenvolver as interfaces web necessárias à criação, ao acompanhamento e à análise das solicitações.
- Implementar mecanismos de rastreabilidade das operações, incluindo o registro de eventos e a apresentação de seu histórico.
- Desenvolver e executar testes unitários e de integração para verificar as regras de negócio e o funcionamento integrado dos componentes da aplicação.

1.2 Escopo e limitações

O trabalho abrange o desenvolvimento do módulo de solicitações e as funcionalidades necessárias ao gerenciamento das operações de alteração cadastral e de classificação dos especialistas no sistema Banco de Especialistas. Estão incluídos os mecanismos de registro, acompanhamento, decisão e reversão das solicitações, bem como os recursos de persistência e rastreabilidade associados a esse fluxo.

Não fazem parte do escopo a redefinição dos critérios de pontuação dos especialistas, a modificação do processo seletivo, a substituição do sistema de avaliação existente e a implementação de notificações por e-mail. Também não é contemplado o tratamento de múltiplas solicitações simultâneas para um mesmo especialista. Essas limitações delimitam as responsabilidades da solução desenvolvida e podem orientar evoluções futuras do sistema.

1.3 Procedimentos metodológicos

O trabalho caracteriza-se como um relato técnico de natureza aplicada e caráter descritivo, desenvolvido no contexto de um estágio supervisionado em desenvolvimento de software. Sua execução compreendeu o estudo da arquitetura e das regras de negócio do sistema, o levantamento e a análise dos requisitos, a modelagem dos componentes necessários, a implementação das funcionalidades e a realização de testes.

A verificação técnica envolveu testes unitários e de integração, destinados a avaliar o comportamento das funcionalidades implementadas e a interação entre os componentes da aplicação. Também foi verificado o funcionamento integrado do front-end e do back-end em ambiente de desenvolvimento conteinerizado com Docker. As atividades foram conduzidas no

contexto da arquitetura e das tecnologias já adotadas pelo projeto, considerando suas restrições e regras de negócio.

As atividades do módulo foram organizadas no OpenProject (OpenProject, 2026), com issues, responsáveis e prazos, o que apoiou a priorização das tarefas e o acompanhamento do progresso ao longo do estágio.

1.4 Contexto institucional

O estágio foi realizado no âmbito do Polo de Inovação do Instituto Federal da Paraíba, estrutura associada à Agência de Inovação da instituição (NEO-IFPB). O Polo atua no desenvolvimento de iniciativas de Pesquisa, Desenvolvimento e Inovação (PD&I), aproximando a instituição de demandas tecnológicas do setor produtivo e da sociedade.

As atividades técnicas foram desenvolvidas no Laboratório Assert, ambiente vinculado ao Polo de Inovação no qual são realizadas atividades de desenvolvimento e manutenção de sistemas computacionais para projetos institucionais. Nesse contexto, o estágio proporcionou a participação nas etapas de análise, implementação, integração e verificação de funcionalidades de uma aplicação web existente.

1.5 Organização do relatório

Além desta introdução, o relatório está organizado em três capítulos. O capítulo 2 apresenta os conceitos e as tecnologias que fundamentam o módulo. O capítulo 3 descreve as atividades do estágio, da análise dos requisitos à implementação e aos testes. O capítulo 4 reúne as considerações finais, com a síntese dos resultados, as limitações e as possibilidades de evolução do trabalho.

2 EMBASAMENTO TÉORICO

Este capítulo apresenta, de forma clara, os conceitos e as tecnologias que fundamentam as atividades descritas neste relatório. Como o estágio teve por foco o desenvolvimento de um módulo de solicitações com a rastreabilidade, auditoria e reversão para o sistema Banco de Especialistas, é dada ênfase ao *back-end* (Flask, SQLAlchemy e PostgreSQL) e aos conceitos de auditoria e versionamento de dados. As demais tecnologias (Vue.js, Nuxt, Pytest, Git e Docker) são abordadas como elementos de apoio ao desenvolvimento, à validação e à execução do módulo.

2.1 Arquitetura Web Cliente-Servidor

A arquitetura cliente-servidor organiza um sistema distribuído em provedores de serviços e requisitantes (TANENBAUM; van Steen, 2007). Nesse contexto, a comunicação orientada pelo estilo REST e pelo protocolo HTTP define interações sem estado, nas quais cada requisição é independente e deve carregar as informações necessárias ao seu processamento (FIELDING; NOTTINGHAM; RESCHKE, 2022).

Essa base teórica fundamenta a separação de responsabilidades do módulo: isolar a máquina de estados e as regras de persistência no servidor garante que a integridade dos dados e as transições de status da solicitação não dependam do cliente. Dessa forma, as aplicações em Vue.js e Nuxt atuam exclusivamente na apresentação e na coleta de informações, consumindo a API REST para criar, acompanhar e analisar as requisições.

2.2 Flask

O Flask foi utilizado na API do módulo por permitir incorporar suas rotas à aplicação existente sem impor uma reorganização do sistema. Os *blueprints* agruparam as operações de criação, análise, acompanhamento e reversão das solicitações em um componente isolado, preservando as demais funcionalidades e facilitando a manutenção do módulo (PALLETS, 2024).

2.3 Persistência de dados com PostgreSQL e SQLAlchemy

O módulo utiliza PostgreSQL para armazenar solicitações, alterações cadastrais e registros de auditoria. A persistência dessas informações precisa manter o cadastro e o histórico coerentes, sobretudo quando uma decisão altera mais de uma entidade.

2.3.1 PostgreSQL

O PostgreSQL foi adequado por oferecer transações para as operações compostas do fluxo. Na aprovação, a mudança de categoria, a atualização da pontuação, a gravação do *snapshot* do estado anterior e a atualização do status da solicitação devem ser confirmadas em conjunto; se uma etapa falhar, nenhuma alteração deve permanecer aplicada (POSTGRESQL GLOBAL DEVELOPMENT GROUP, 2024).

2.3.2 SQLAlchemy

O SQLAlchemy foi adotado como ORM para coordenar as alterações de entidades relacionadas em uma única unidade de trabalho (BAYER, 2012). A sessão acumula as mudanças e, no *commit*, efetiva-as na transação; em caso de falha, o *rollback* descarta o conjunto. Assim, a aprovação não deixa a categoria atualizada sem o *snapshot* correspondente ou sem o novo status da solicitação.

2.4 Validação de Dados com Marshmallow

Na fronteira da API, os *schemas* do Marshmallow validam os campos e formatos recebidos antes que os dados cheguem às regras de negócio (MARSHMALLOW DEVELOPERS, 2024). Ao declarar os formatos esperados para as solicitações e suas respostas, também estabelecem o contrato consumido pelas interfaces de criação, acompanhamento e análise do módulo.

2.5 Auditoria, Rastreabilidade e Versionamento de Dados

Esta seção reúne os conceitos que sustentam a principal contribuição do módulo: a capacidade de registrar, consultar e desfazer alterações cadastrais com garantias de rastreabilidade.

2.5.1 Auditoria e rastreabilidade

Auditoria, no contexto de sistemas de informação, é o registro sistemático de quem realizou determinada operação, quando, sobre qual dado e com qual resultado. A rastreabilidade é a propriedade que permite reconstituir o histórico de uma informação a partir desses registros. Fowler (FOWLER, 2004) descreve o padrão *Audit Log* como um registro de alterações feitas nos dados, com o objetivo de permitir verificar o que ocorreu e, quando necessário, compreender a origem de um estado atual. Em sistemas que envolvem decisões de classificação e seleção de pessoas, como o Banco de Especialistas, esse registro contribui para a transparência e para a responsabilização dos agentes envolvidos.

O fluxo anterior ao módulo, baseado em e-mail e execução manual de SQL, ilustra a ausência dessas propriedades: não havia registro formal do solicitante, da justificativa, do

responsável pela decisão ou do estado anterior do cadastro. Ao formalizar a solicitação como uma entidade persistente, com status, autoria e datas, o sistema passa a produzir esse registro como efeito natural do próprio fluxo de trabalho.

2.5.2 Fluxo de aprovação com máquina de estados

Uma solicitação percorre um ciclo de vida bem definido, que pode ser modelado como uma máquina de estados finitos: um conjunto de estados (por exemplo, pendente, aprovada e rejeitada) e de transições permitidas entre eles (SOMMERVILLE, 2011). Explicitar as transições válidas evita estados incoerentes: uma solicitação rejeitada não pode ser aprovada sem nova análise, e somente uma solicitação aprovada pode ser revertida. A validação dessas regras no servidor garante que o fluxo seja respeitado independentemente da interface utilizada.

2.5.3 Snapshot e reversão

Para possibilitar a reversão de uma alteração, é necessário preservar o estado anterior dos dados. O conceito de *snapshot* (instantâneo) consiste em capturar e armazenar o estado de um objeto em determinado momento. Ele se aproxima do padrão *Memento*, descrito por Gamma et al. (GAMMA et al., 1994), cujo propósito é capturar o estado interno de um objeto, sem violar seu encapsulamento, para que ele possa ser restaurado posteriormente.

No módulo, a entidade *Snapshot* registra o estado relevante do especialista (como categoria e pontuação) antes da aplicação de uma alteração aprovada. A reversão consiste em restaurar esses valores, mantendo a consistência entre categoria e pontuação. Cabe distinguir essa abordagem da simples exclusão do registro de histórico: a reversão é, ela mesma, uma operação registrada, de modo que o histórico permanece completo e a operação inversa pode ser auditada. Esse comportamento é análogo à ideia de transações compensatórias, em que o efeito de uma operação já efetivada é desfeito por uma nova operação, e não pela remoção do passado.

Existem abordagens mais gerais para o versionamento de dados, como o *Event Sourcing*, em que o estado é derivado de uma sequência imutável de eventos (FOWLER, 2005; KLEPPMANN, 2017). O módulo adota uma solução mais simples e proporcional ao escopo, combinando *snapshots* do estado anterior com um registro de eventos, e o conceito de *Event Sourcing* é aqui mencionado como referência conceitual para a ideia de histórico imutável.

2.5.4 Timeline e registro de eventos

A *timeline* é a apresentação ordenada cronologicamente dos eventos associados a uma entidade. Cada evento (criação, aprovação, rejeição, reversão) é registrado com seu tipo, autor, data e informações complementares, e a interface os exibe como uma linha do tempo. Trata-se de uma forma de tornar a auditoria acessível ao usuário final, e não apenas ao administrador do banco de dados.

2.6 Vue.js e Nuxt

Vue.js e Nuxt foram utilizados na construção das interfaces do módulo (VUE.JS, 2024; NUXT, 2024). As páginas de análise e acompanhamento consomem a API para apresentar o estado e o histórico das solicitações; a interface administrativa também permite ao responsável aprovar, rejeitar ou reverter uma solicitação.

2.7 Verificação das Transições com Pytest

O Pytest foi empregado para verificar as regras de negócio do módulo, em especial as transições permitidas da máquina de estados: criação, aprovação, rejeição e reversão (HOLGER KREKEL AND PYTEST DEVELOPERS, 2024). Os testes também verificaram os efeitos dessas transições sobre a categoria, a pontuação e o histórico da solicitação.

2.8 Ambiente Reprodutível de Testes com Docker

O Docker foi utilizado para reproduzir o ambiente necessário aos testes de integração, incluindo a API e o PostgreSQL, de modo que a validação dos fluxos ocorresse sob uma configuração consistente entre execuções (DOCKER, 2024).

2.9 Controle de Versão e Gestão de Configuracao

A gestão de configuração de software compreende a identificação, o controle e o registro das alterações realizadas nos artefatos de um projeto ao longo do tempo, de modo que seja possível reconstituir versões anteriores e identificar a origem de cada mudança (SOMMERVILLE, 2011). O controle de versão é o mecanismo que concretiza esse registro sobre o código-fonte: cada alteração é capturada em um commit, com autor, data e descrição, e o histórico resultante permite recuperar estados anteriores do sistema.

O módulo utilizou Git (**??**) para esse fim, com ramificações (branches) por funcionalidade, o que isolou o desenvolvimento do módulo das demais partes do Banco de Especialistas até que as alterações fossem verificadas. Os repositórios foram hospedados no GitLab (GitLab, 2026), adotado como padrão pelo Laboratório Assert para os projetos do Polo de Inovação, e nele foi configurada uma pipeline de integração contínua que executa a suíte de testes (Pytest) a cada envio, impedindo que alterações que quebrem as regras de negócio sejam incorporadas à base principal.

Cabe observar que o controle de versão atua sobre o código-fonte, e não sobre os dados da aplicação. A rastreabilidade das alterações cadastrais dos especialistas, tratada na seção 2.5, é um problema distinto, resolvido no domínio da aplicação por meio de solicitações, snapshots e registro de eventos.

2.10 Síntese do Capítulo

As tecnologias e os conceitos apresentados compõem a base sobre a qual o módulo de solicitações foi desenvolvido. A arquitetura cliente-servidor e o estilo REST definem a forma de comunicação entre as interfaces e a API; o Flask estrutura a aplicação, o SQLAlchemy e o PostgreSQL garantem a persistência transacional, e o Marshmallow assegura a validação e o contrato dos dados. Os conceitos de auditoria, máquina de estados, *snapshot* e registro de eventos sustentam os requisitos de rastreabilidade e reversão que motivaram o estágio. Por fim, Vue.js e Nuxt viabilizam as interfaces, o Pytest garante a verificação das regras de negócio, o Git organiza a colaboração e o Docker fornece o ambiente reprodutível de execução. O próximo capítulo descreve como esses elementos foram aplicados nas atividades realizadas.

3 ATIVIDADES REALIZADAS

Este capítulo apresenta as principais atividades realizadas no decorrer do estudo e do desenvolvimento da funcionalidade de alteração de categoria de especialistas. A partir da análise do sistema existente e do levantamento das regras de negócio, foi possível compreender o contexto institucional, os fluxos de operação e os requisitos necessários para a implementação de uma solução integrada, auditável e compatível com a arquitetura atual.

3.1 Estudo da Arquitetura e Levantamento de Requisitos

O primeiro passo para o desenvolvimento da funcionalidade consiste na compreensão do sistema Banco de Especialistas, de sua finalidade institucional e de sua organização técnica. A plataforma permite o cadastramento e a classificação de profissionais interessados em participar de projetos de pesquisa, desenvolvimento e inovação vinculados ao IFPB.

Nesse contexto, foram estudadas as funcionalidades relacionadas ao cadastro dos especialistas, à organização por categorias e aos critérios de classificação utilizados pelo sistema. Também foram examinados os componentes da aplicação e suas interações, com o objetivo de identificar os pontos de integração necessários ao desenvolvimento do novo módulo.

De certo modo, um dos princípios que foram usados é o Additive Change, no gerenciamento de APIs e controladores de código: a prática de evoluir o sistema apenas adicionando novas rotas, campos ou módulos, visando a compatibilidade com o código anterior. A escolha de evitar grandes alterações ao código veio da reflexão baseada na forma como o código antigo foi escrito, seguindo uma arquitetura MVC com a camada de controller sobrecarregada de funções, aproximando-se mais de uma versão procedural com baixa manutenção e abstração. Os módulos recém-construídos já foram melhor desenvolvidos com base nos pontos anteriormente analisados.

3.2 Análise das Regras de Negócio

O sistema possui um modelo de regras de negócio relacionadas à classificação de especialistas e à alteração de seus dados cadastrais. Condições essas que deveriam ser respeitadas pela nova funcionalidade, bem como os efeitos das operações sobre os registros existentes.

Foram examinados os critérios relacionados à mudança de categoria, à atualização da pontuação e às responsabilidades dos usuários envolvidos no processo. Também foi necessário distinguir as operações de solicitação das ações de aprovação e rejeição, uma vez que essas etapas possuem finalidades e consequências distintas. O levantamento dessas regras serviu de base para definir o comportamento esperado do módulo e evitar que as novas operações comprometessem as restrições já estabelecidas pelo sistema.

Com base no sistema atual, os principais desafios para realizar a transição de categoria de um especialista começam pelo fato de que a categoria de um especialista não estar armazenada no modelo Specialist, mas sim no User.

3.3 Entendimento do Fluxo Antigo

Antes do desenvolvimento do módulo, as solicitações de mudanças de categoria eram realizadas por meio de correio eletrônico. Após o recebimento do pedido, a alteração dependia da intervenção de um membro da equipe responsável, que executava manualmente as operações necessárias diretamente no banco de dados por meio de uma QUERY SQL reutilizável.

Esse procedimento estabelece uma dependência entre a solicitação do especialista e a disponibilidade de um responsável com acesso técnico à base de dados. Além disso, o acompanhamento do pedido e a consulta ao histórico das alterações dependiam dos registros e das comunicações existentes fora de um fluxo centralizado e auditável na aplicação.

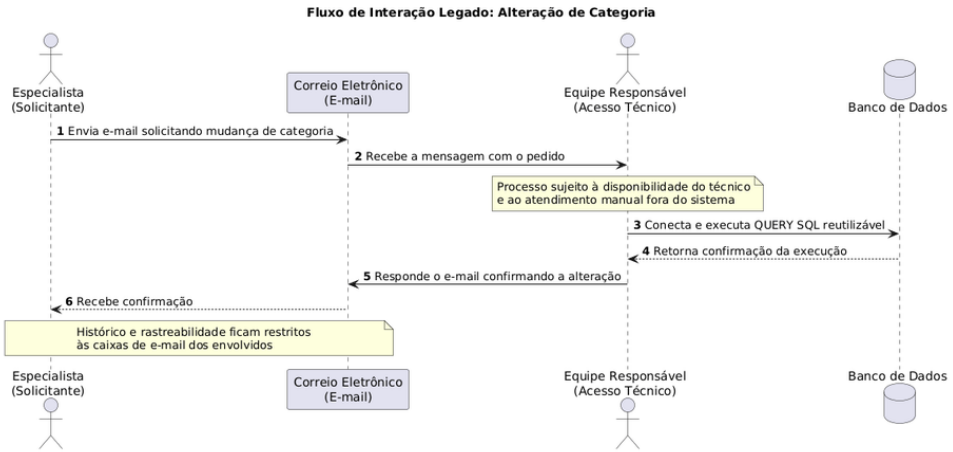


Figura 1 – Fluxo de alteração de categoria de especialistas antes do desenvolvimento do módulo.

A análise desse processo permitiu identificar a necessidade de incorporar ao sistema mecanismos para registrar as solicitações, formalizar as decisões e acompanhar as alterações realizadas.

3.4 Desenvolvimento do Módulo de Solicitações

3.4.1 Endpoints

3.4.2 Integração com o banco de dados

3.4.3 Aprovação e Rejeição

3.4.4 Mudança de categoria

3.4.5 Atualização de pontuação

3.5 Implementação do Sistema de Snapshot e Auditoria

3.5.1 Entidade Snapshot

3.5.2 JSONB

3.5.3 Reversão

3.5.4 Timeline

3.5.5 Rastreabilidade

3.6 Desenvolvimento das Interfaces Web

3.6.1 Telas Criadas

3.6.2 Componentes Vue

3.6.3 Integração frontend/backend

3.7 Testes e Validação

3.7.1 Pytest

3.7.2 Pré-configuração

3.7.3 Testes unitários

3.7.4 Testes de integração

3.7.5 Processo de Integração Contínua

3.8 Levantamento de Requisitos das Funcionalidades

4 CONSIDERAÇÕES FINAIS

Cada capítulo deve conter uma pequena introdução (tipicamente, um ou dois parágrafos) que deve deixar claro o objetivo e o que será discutido no capítulo, bem como a organização do capítulo.

Referências

BAYER, M. Sqlalchemy. In: BROWN, A.; WILSON, G. (Ed.). **The Architecture of Open Source Applications, Volume II: Structure, Scale, and a Few More Fearless Hacks**. Lulu.com, 2012. Disponível em: <https://aosabook.org/en/v2/sqlalchemy.html>. Acesso em: 25 out. 2026. Citado na página 5.

DOCKER. **Docker Documentation**. [S.l.], 2024. Disponível em: <https://docs.docker.com/>. Acesso em: 26 out. 2026. Citado na página 7.

FIELDING, R. T.; NOTTINGHAM, M.; RESCHKE, J. **HTTP Semantics**. [S.l.], 2022. Disponível em: <https://www.rfc-editor.org/info/rfc9110>. Acesso em: 25 out. 2026. Citado na página 4.

FOWLER, M. **Audit Log**. 2004. Martin Fowler's Bliki. Disponível em: <https://martinfowler.com/eaaDev/AuditLog.html>. Acesso em: 25 out. 2026. Citado na página 5.

FOWLER, M. **Event Sourcing**. 2005. Martin Fowler's Bliki. Disponível em: <https://martinfowler.com/eaaDev/EventSourcing.html>. Acesso em: 25 out. 2026. Citado na página 6.

GAMMA, E. et al. **Design Patterns: Elements of Reusable Object-Oriented Software**. Reading, Massachusetts: Addison-Wesley, 1994. Citado na página 6.

GitLab. **GitLab Documentation**. 2026. <https://docs.gitlab.com/>. Acesso em: 25 out. 2026. Citado na página 7.

HOLGER KREKEL AND PYTEST DEVELOPERS. **pytest: helps you write better programs**. [S.l.], 2024. Disponível em: <https://docs.pytest.org/>. Acesso em: 25 out. 2026. Citado na página 7.

KLEPPMANN, M. **Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems**. Sebastopol, CA: O'Reilly Media, 2017. Citado na página 6.

MARSHMALLOW DEVELOPERS. **marshmallow: simplified object serialization**. [S.l.], 2024. Disponível em: <https://marshmallow.readthedocs.io/>. Acesso em: 25 out. 2026. Citado na página 5.

NUXT. **Nuxt: The Intuitive Web Framework**. [S.l.], 2024. Disponível em: <https://nuxt.com/docs/getting-started/introduction>. Acesso em: 25 out. 2026. Citado na página 7.

OpenProject. **OpenProject Documentation**. 2026. <https://www.openproject.org/docs/>. Acesso em: 25 out. 2026. Citado na página 3.

PALLETS. **Flask Documentation**. [S.l.], 2024. Disponível em: <https://flask.palletsprojects.com/>. Acesso em: 25 out. 2026. Citado na página 4.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 16 Documentation**. [S.l.], 2024. Disponível em: <https://www.postgresql.org/docs/16/>. Acesso em: 25 out. 2026. Citado na página 5.

SOMMERVILLE, I. **Engenharia de Software**. 9. ed. São Paulo: Pearson Prentice Hall, 2011. Citado 2 vezes nas páginas 6 e 7.

TANENBAUM, A. S.; van Steen, M. **Sistemas Distribuídos: Princípios e Paradigmas**. 2. ed. São Paulo: Pearson Prentice Hall, 2007. Citado na página 4.

VUE.JS. **Vue.js: The Progressive JavaScript Framework**. [S.l.], 2024. Disponível em: <https://vuejs.org/guide/introduction.html>. Acesso em: 25 out. 2026. Citado na página 7.

Apêndices

APÊNDICE A – Nome do apêndice

Lembre-se que a diferença entre apêndice e anexo diz respeito à autoria do texto e/ou material ali colocado.

Caso o material ou texto suplementar ou complementar seja de sua autoria, então ele deverá ser colocado como um apêndice. Porém, caso a autoria seja de terceiros, então o material ou texto deverá ser colocado como anexo.

Caso seja conveniente, podem ser criados outros apêndices para o seu trabalho acadêmico. Basta recortar e colar este trecho neste mesmo documento. Lembre-se de alterar o "label" do apêndice.

Não é aconselhável colocar tudo que é complementar em um único apêndice. Organize os apêndices de modo que, em cada um deles, haja um único tipo de conteúdo. Isso facilita a leitura e compreensão para o leitor do trabalho.

APÊNDICE B – Nome do outro apêndice

conteúdo do novo apêndice

Anexos

ANEXO A – Nome do anexo

Lembre-se que a diferença entre apêndice e anexo diz respeito à autoria do texto e/ou material ali colocado.

Caso o material ou texto suplementar ou complementar seja de sua autoria, então ele deverá ser colocado como um apêndice. Porém, caso a autoria seja de terceiros, então o material ou texto deverá ser colocado como anexo.

Caso seja conveniente, podem ser criados outros anexos para o seu trabalho acadêmico. Basta recortar e colar este trecho neste mesmo documento. Lembre-se de alterar o "label" do anexo.

Organize seus anexos de modo a que, em cada um deles, haja um único tipo de conteúdo. Isso facilita a leitura e compreensão para o leitor do trabalho. É para ele que você escreve.

ANEXO B – Nome do outro anexo

conteúdo do outro anexo